

Trabajo de Segundo Corte

Análisis interno de ESM-2: un Transformer científico para secuencias de proteínas

Documento guía para estudiantes de último semestre de pregrado

Enfoque del trabajo

El centro de la actividad no es entrenar un modelo, hacer RAG ni hacer fine-tuning. El centro es **entender cómo opera un transformer científico preentrenado**, ejecutar ESM-2, inspeccionar su arquitectura, identificar sus componentes en código fuente, visualizar representaciones internas y explicar con rigor qué significan tokens, embeddings, hidden states, atención multi-cabeza y salidas del modelo.

Docente: _____

Fecha de asignación: _____

Fecha de entrega: _____

Modalidad sugerida: Grupos de 2 a 3 estudiantes; defensa oral individual.

Producto principal: Notebook reproducible + informe técnico + sustentación.

Índice

1. Síntesis de la consigna	1
1.1. Qué se pide y qué no se pide	1
2. Marco teórico inicial: qué es un Transformer	1
2.1. Tokens, embeddings y representaciones contextualizadas	1
2.2. Self-attention: el mecanismo central	2
2.3. Tipos principales de transformers	2
3. Qué es ESM-2	3
3.1. ESM-2 no es un chatbot	3
3.2. Mínimo biológico necesario	4
3.3. Flujo interno de ESM-2	4
3.4. Relación con ESMFold	4
4. Usos reales de ESM-2	5
4.1. Extracción de embeddings de proteínas	5
4.2. Análisis de mutaciones	5
4.3. Apoyo a predicción de estructura	5
4.4. Anotación funcional y clasificación posterior	5
4.5. Limitaciones	5
5. Trabajo de Segundo Corte	5
5.1. Título del trabajo	6
5.2. Propósito evaluativo	6
5.3. Modelo obligatorio	6
5.4. Preguntas orientadoras	6
6. Actividades obligatorias	6
6.1. Actividad 1: marco teórico propio	7
6.2. Actividad 2: ejecución de ESM-2 preentrenado	7
6.3. Actividad 3: inspección del código fuente	7
6.4. Actividad 4: visualización de embeddings y hidden states	8
6.5. Actividad 5: matrices de atención	8
6.6. Actividad 6: masked language modeling	9
6.7. Actividad 7: usos reales y límites	9

7. Evidencias mínimas que debe contener el notebook	9
7.1. Código base sugerido	10
7.2. Visualización de una matriz de atención	10
7.3. Masked language modeling con top-k	11
8. Entregables	11
8.1. Entregable 1: notebook reproducible	11
8.2. Entregable 2: informe técnico	11
8.3. Entregable 3: defensa oral individual	12
9. Rúbrica de evaluación	12
10. Criterios de no aprobación	12
11. Preguntas de defensa	13
12. Cronograma sugerido	13
13. Anexo A: plantilla de estructura del informe	13
14. Anexo B: secuencias de ejemplo	14
15. Anexo C: lectura mínima recomendada	14

1 Síntesis de la consigna

Objetivo general

Analizar ESM-2 como un **transformer encoder-only científico** aplicado a secuencias de aminoácidos, demostrando cómo una secuencia proteica se transforma en tokens, embeddings iniciales, representaciones contextualizadas, matrices de atención y salidas útiles para análisis de datos científicos.

1.1 Qué se pide y qué no se pide

Sí se pide	No se pide
Usar un modelo ESM-2 ya entrenado, preferiblemente <code>facebook/esm2_t6_8M_UR50D</code> .	Entrenar ESM-2 desde cero.
Ejecutar inferencia sobre secuencias de proteínas.	Hacer fine-tuning, LoRA, PEFT o ajuste supervisado.
Extraer tokens, embeddings, hidden states, logits y atenciones si el entorno lo permite.	Construir un transformer completo desde cero.
Leer e identificar en código fuente los componentes de la arquitectura.	Hacer RAG, agente conversacional o chatbot.
Visualizar matrices de atención y comparar secuencias original, mutada y alterada.	Formular conclusiones biomédicas clínicas o experimentales.
Explicar usos reales y limitaciones de ESM-2.	Presentar solo teoría sin ejecución ni evidencia computacional.

Regla de diseño pedagógico

Este trabajo no evalúa quién hace la interfaz más llamativa. Evalúa si el estudiante puede explicar con evidencia computacional qué ocurre dentro de un transformer científico preentrenado.

2 Marco teórico inicial: qué es un Transformer

El transformer fue propuesto como una arquitectura basada en mecanismos de atención, prescindiendo de recurrencia y convolución para tareas de secuencia (Vaswani et al., 2017). Su núcleo es una idea general: convertir unidades de información en vectores y calcular relaciones entre esas unidades mediante atención.

2.1 Tokens, embeddings y representaciones contextualizadas

Un modelo transformer no procesa directamente palabras, imágenes, aminoácidos o documentos. Primero convierte la entrada en unidades discretas llamadas **tokens**. En lenguaje natural, un token puede ser una palabra o subpalabra. En una proteína, un token puede ser un aminoácido. En visión, un token puede ser un parche de imagen (Dosovitskiy et al., 2020).

Luego cada token se convierte en un vector numérico: el **embedding inicial**. Ese vector ubica la unidad de información en un espacio matemático. Después de pasar por capas transformer, el vector se convierte en una **representación contextualizada**: ya no representa solo el token aislado, sino el token dentro de la secuencia completa.

Diferencia crítica

Embedding inicial: ficha numérica inicial del token.

Hidden state o representación contextualizada: vector después de atención y capas transformer; incorpora información de otros tokens.

2.2 Self-attention: el mecanismo central

La atención calcula qué tan relevante es cada token para interpretar otro token. En su forma simplificada:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Donde:

- Q son las consultas: qué busca cada token.
- K son las claves: cómo se anuncia cada token ante los demás.
- V son los valores: qué información aporta cada token.

En la analogía de clase:

Analogía de la escena del caso

La secuencia es una escena del caso. Cada token es un testigo inteligente. Cada cabeza de atención es un detective que cruza declaraciones. La matriz de atención es la tabla donde se registra cuánto debe escuchar cada testigo a los demás para entender su papel en la escena.

2.3 Tipos principales de transformers

Tipo	Cómo atiende	Uso típico	Ejemplos
Encoder-only	Atención bidireccional: todos los tokens pueden mirar a todos los tokens válidos de la entrada.	Comprensión, clasificación, embeddings, masked language modeling.	BERT, ESM-2, ViT en clasificación.
Decoder-only	Atención causal: cada token mira tokens anteriores y a sí mismo, no tokens futuros.	Generación autoregresiva: siguiente token, luego otro, luego otro.	GPT y muchos LLM conversacionales.

Encoder-decoder	Un encoder comprende la entrada completa y un decoder genera la salida condicionada.	Traducción, resumen, transducción secuencial.	Transformer original, T5, BART.
-----------------	--	---	---------------------------------

BERT es un caso canónico de encoder-only bidireccional con entrenamiento por enmascaramiento de tokens (Devlin et al., 2018). GPT, por el contrario, usa un transformer decoder multicapa que produce una distribución sobre tokens objetivo a partir del contexto disponible (Radford et al., 2018). ESM-2 se entiende mejor como un modelo tipo encoder-only aplicado a proteínas.

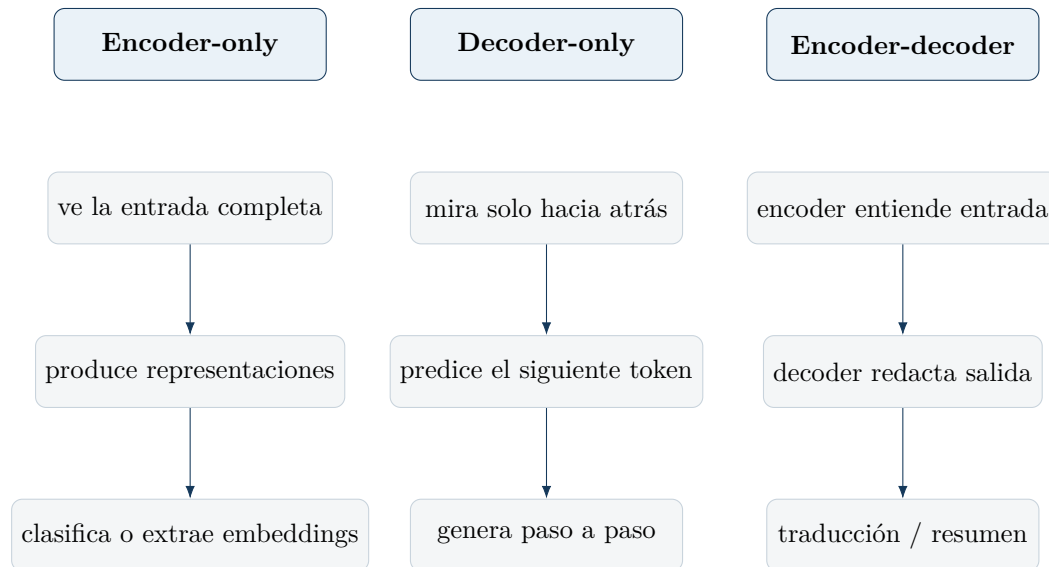


Figura 1: Tres familias de arquitectura transformer y su lógica de uso.

3 Qué es ESM-2

ESM significa *Evolutionary Scale Modeling*. ESM-2 es un modelo de lenguaje de proteínas desarrollado por Meta AI/FAIR para procesar secuencias de aminoácidos mediante transformers. El repositorio oficial de ESM contiene código y pesos preentrenados para modelos de lenguaje de proteínas, incluyendo ESM-2, ESMFold, MSA Transformer, ESM-1v y ESM-IF1 (Meta Fundamental AI Research Protein Team, 2026). Hugging Face también provee documentación y pesos preentrenados para estos modelos (Hugging Face, 2026a).

3.1 ESM-2 no es un chatbot

ESM-2 no está diseñado para conversar. Es un transformer científico que recibe secuencias de aminoácidos. Su salida principal son representaciones matemáticas de residuos y proteínas, además de logits para tareas de masked language modeling. El model card de `facebook/esm2_t6_8M_UR50D` lo describe como un modelo de proteínas entrenado con objetivo de masked language modeling para tareas que toman secuencias proteicas como entrada (Hugging Face and Meta AI, 2026).

Definición operativa

ESM-2 es un **transformer encoder-only de proteínas** que puede usarse como **modelo de embeddings científicos**. Arquitectónicamente es un transformer; funcionalmente se usa con frecuencia para obtener vectores por aminoácido o por proteína completa.

3.2 Mínimo biológico necesario

No se requiere conocimiento biomédico avanzado. Para este trabajo basta con comprender:

1. Una proteína puede representarse como una secuencia de aminoácidos.
2. Cada aminoácido se codifica con una letra: A, C, D, E, F, G, H, I, K, L, M, N, P, Q, R, S, T, V, W, Y.
3. El orden importa: las mismas letras en otro orden no describen la misma secuencia funcional.
4. Una mutación puntual es un cambio de una letra por otra en una posición.
5. El modelo aprende patrones estadísticos de secuencias; no reemplaza validación experimental.

3.3 Flujo interno de ESM-2

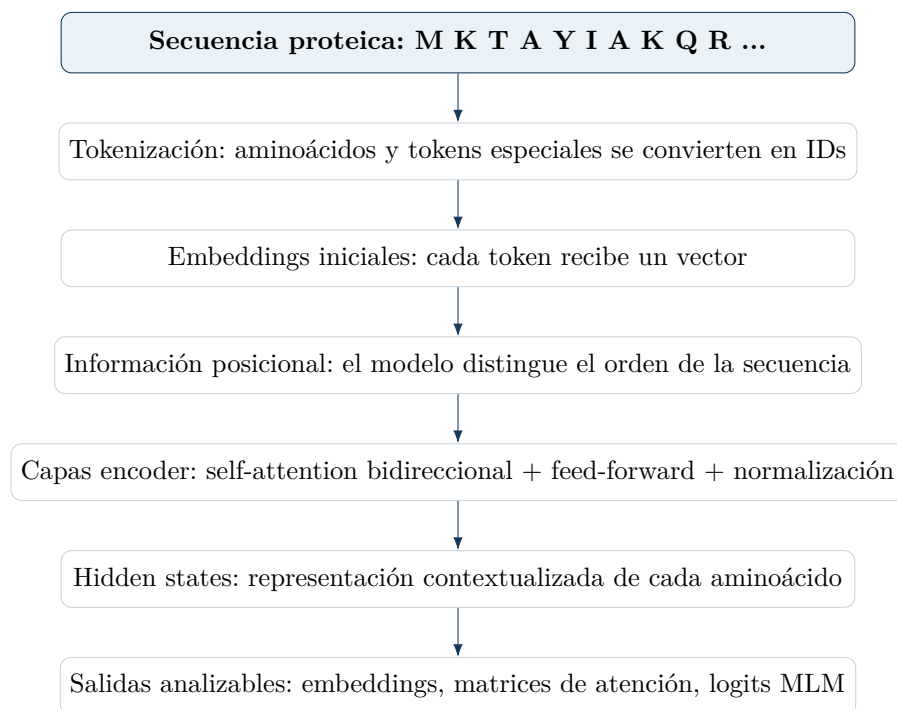


Figura 2: Flujo didáctico de ESM-2 para una secuencia de aminoácidos.

3.4 Relación con ESMFold

ESM-2 y ESMFold no son exactamente lo mismo. ESM-2 es el modelo de lenguaje de proteínas que produce representaciones. ESMFold usa representaciones de ESM-2 y módulos adicionales para predecir

estructura tridimensional a partir de secuencia. El trabajo de Lin et al. (2023) reporta inferencia directa de estructura proteica a nivel atómico usando modelos de lenguaje de proteínas escalados.

4 Usos reales de ESM-2

4.1 Extracción de embeddings de proteínas

El uso más directo es convertir una proteína en vectores. Se pueden obtener vectores por aminoácido y un vector global, por ejemplo promediando representaciones de residuos:

$$\mathbf{h}_{\text{proteína}} = \frac{1}{n} \sum_{i=1}^n \mathbf{h}_i$$

Esto permite comparar proteínas mediante similitud coseno, hacer agrupamiento, detectar secuencias cercanas o construir índices vectoriales.

4.2 Análisis de mutaciones

Una secuencia original y una secuencia mutada pueden compararse por cambios en embeddings, logits o representaciones de regiones cercanas a la mutación. Esto no prueba por sí solo un efecto biológico, pero ayuda a formular hipótesis y priorizar análisis.

4.3 Apoyo a predicción de estructura

ESMFold aprovecha ESM-2 para predicción estructural desde la secuencia. Para este trabajo no se exige usar ESMFold, pero es importante que el estudiante entienda que ESM-2 puede servir como base para modelos científicos posteriores.

4.4 Anotación funcional y clasificación posterior

Los embeddings de ESM-2 pueden alimentar clasificadores o modelos posteriores para tareas como clasificación funcional, localización subcelular, similitud entre familias proteicas o búsqueda de proteínas relacionadas. En este trabajo no se entrenará un clasificador, pero sí se debe explicar el flujo conceptual.

4.5 Limitaciones

Lo que no debe afirmarse

No se debe decir que ESM-2 descubre medicamentos automáticamente, reemplaza experimentos de laboratorio o demuestra causalidad biológica mediante mapas de atención. ESM-2 produce representaciones computacionales útiles, pero la interpretación científica requiere validación, contexto y, en muchos casos, experimentación.

5 Trabajo de Segundo Corte

5.1 Título del trabajo

Abrir la caja negra de ESM-2: tokens, embeddings, atención y representaciones contextualizadas en un transformer científico

5.2 Propósito evaluativo

El trabajo busca garantizar que el estudiante comprenda el funcionamiento de un transformer real a partir de evidencia computacional. No basta con ejecutar un modelo; el estudiante debe explicar qué se ejecuta, dónde ocurre en el código y cómo se interpreta cada salida.

5.3 Modelo obligatorio

Se recomienda usar:

`facebook/esm2_t6_8M_UR50D`

Motivos:

- Tiene tamaño bajo para docencia: aproximadamente 8 millones de parámetros.
- Está disponible como modelo preentrenado.
- Permite procesar secuencias de proteínas sin entrenamiento adicional.
- Puede devolver `hidden_states` y `attentions` si se configura adecuadamente.

Se permite usar `esm2_t12_35M_UR50D` si el equipo cuenta con recursos, pero no se evaluará mayor nota por usar un modelo más grande.

5.4 Preguntas orientadoras

El informe debe responder, como mínimo, las siguientes preguntas:

1. ¿Qué es ESM-2 y por qué se considera un transformer científico?
2. ¿Por qué ESM-2 es encoder-only y no decoder-only como GPT?
3. ¿Qué es un token en una secuencia proteica?
4. ¿Qué diferencia hay entre embedding inicial y hidden state contextualizado?
5. ¿Qué representan Q , K y V en self-attention?
6. ¿Qué representa una matriz de atención de forma $(L \times L)$?
7. ¿Qué diferencia se observa entre secuencia original, mutada y permutada?
8. ¿Qué usos reales tiene ESM-2 en ciencia de datos aplicada a proteínas?
9. ¿Qué limitaciones tiene interpretar mapas de atención como explicación causal?

6 Actividades obligatorias

6.1 Actividad 1: marco teórico propio

Cada grupo debe elaborar un marco teórico breve, con citas, sobre:

- transformer original;
- encoder-only, decoder-only y encoder-decoder;
- embeddings y representaciones contextualizadas;
- atención multi-cabeza;
- masked language modeling;
- ESM-2 como modelo de lenguaje de proteínas.

Debe estar escrito por el grupo. Se permite usar IA generativa como apoyo, pero debe declararse y revisarse críticamente.

6.2 Actividad 2: ejecución de ESM-2 preentrenado

El grupo debe ejecutar ESM-2 con al menos tres secuencias:

1. **Secuencia original:** una proteína corta real o una secuencia suministrada por el profesor.
2. **Secuencia mutada:** la misma secuencia con una mutación puntual documentada por el grupo.
3. **Secuencia alterada:** una permutación, inversión parcial o modificación severa usada como control.

Debe reportarse:

- longitud de secuencia;
- tokens producidos;
- IDs de tokens;
- forma de `last_hidden_state`;
- número de capas y cabezas del modelo usado;
- forma de las atenciones si están disponibles.

6.3 Actividad 3: inspección del código fuente

El grupo debe identificar en código fuente los componentes principales. Puede usar el repositorio de Hugging Face Transformers o el repositorio oficial de Meta.

Debe documentarse una tabla como esta:

Componente	Dónde se identifica en código	Qué hace
Tokenizer / vocabulario	Clase o archivo usado para tokenizar ESM	Convierte aminoácidos y tokens especiales en IDs.
Embeddings	Módulo de embeddings de ESM	Convierte IDs en vectores iniciales.
Self-attention	Clase o función de atención	Calcula Q , K , V , pesos de atención y salida.

Multi-head attention	Proyecciones y combinación de cabezas	Procesa varias relaciones en paralelo.
LayerNorm y residual	Bloque transformer	Estabiliza y preserva información entre subcapas.
Feed-forward	MLP por posición	Transforma cada representación contextual.
MLM head	Cabeza de lenguaje enmascarado	Produce logits para predecir tokens ocultos.

No basta con citar el nombre del archivo

El grupo debe explicar qué encontró en el código y cómo se conecta con la teoría. Deben incluir fragmentos breves de código o capturas anotadas, sin copiar grandes bloques innecesarios.

6.4 Actividad 4: visualización de embeddings y hidden states

Deben extraer representaciones de la última capa y construir:

1. un embedding por residuo;
2. un embedding global de proteína mediante promedio o estrategia justificada;
3. una comparación de similitud coseno entre secuencia original, mutada y alterada;
4. una proyección 2D simple, por ejemplo PCA, de embeddings por residuo o por secuencia.

La visualización no se evalúa por estética sino por interpretación. Debe quedar claro qué significa cada punto y qué limitaciones tiene.

6.5 Actividad 5: matrices de atención

Si el entorno lo permite, deben usar `output_attentions=True` y visualizar matrices de atención. La documentación de Transformers describe las atenciones como tensores por capa, después del softmax, usados para calcular promedios ponderados en self-attention (Hugging Face, 2026b). La forma esperada es:

(batch, num_heads, sequence_length, sequence_length)

Deben presentar como mínimo:

- una matriz de atención de una capa temprana;
- una matriz de atención de una capa intermedia o profunda;
- dos cabezas distintas de una misma capa;
- comparación cualitativa entre secuencia original y mutada.

Interpretación mínima de una matriz de atención

Una matriz $L \times L$ indica, para una cabeza y una capa, cuánto atiende cada posición de la secuencia a las demás posiciones. Cada fila puede interpretarse como la distribución de atención de un token hacia otros tokens válidos.

Si las atenciones no aparecen

Algunas implementaciones optimizadas pueden no retornar mapas de atención de forma directa. Si `outputs.attentions` aparece como `None`, el grupo debe documentar el intento, revisar la configuración del modelo y probar opciones como cargar el modelo con implementación de atención compatible con retorno explícito de atenciones. Si aun así no se logra, debe entregar una explicación técnica y centrar la visualización en `hidden_states`, logits y lectura del código de atención.

6.6 Actividad 6: masked language modeling

El grupo debe construir un ejemplo con un aminoácido enmascarado y obtener los **top-k** tokens predichos por ESM-2. Debe explicar:

- qué posición fue enmascarada;
- qué tokens candidatos recibió mayor puntaje;
- por qué esto no equivale a “generar una proteína” como GPT genera texto;
- por qué ESM-2 usa contexto bidireccional.

6.7 Actividad 7: usos reales y límites

El informe debe incluir una sección de usos reales, con mínimo cuatro aplicaciones:

- comparación de proteínas mediante embeddings;
- análisis de mutaciones;
- apoyo a predicción de estructura mediante modelos derivados como ESMFold;
- anotación funcional o clasificación posterior;
- búsqueda semántica de proteínas en bases vectoriales;
- priorización de variantes para ingeniería de proteínas.

También debe incluir limitaciones: no causalidad directa desde atención, no sustitución de laboratorio, sesgos de datos y restricciones de cómputo.

7 Evidencias mínimas que debe contener el notebook

El notebook debe ser ejecutable y documentado. Debe incluir al menos:

1. instalación o declaración de entorno;
2. carga del tokenizer y modelo;
3. tokenización de una secuencia;
4. extracción de `last_hidden_state`;
5. extracción de `hidden_states`;
6. intento de extracción de `attentions`;
7. gráfica de atención si está disponible;
8. cálculo de embedding global;
9. similitud coseno entre secuencias;

10. masked language modeling con `top-k`;
11. conclusiones técnicas.

7.1 Código base sugerido

El siguiente código es una guía mínima. El grupo debe adaptarlo, comentarlo y explicar sus salidas.

```
1 import torch
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from transformers import AutoTokenizer, EsmModel, EsmForMaskedLM
5 from sklearn.metrics.pairwise import cosine_similarity
6
7 model_name = "facebook/esm2_t6_8M_UR50D"
8
9 tokenizer = AutoTokenizer.from_pretrained(model_name)
10 model = EsmModel.from_pretrained(model_name)
11 model.eval()
12
13 sequence = "MKTAYIAKQRQISFVKSHFSRQDILD"
14 inputs = tokenizer(sequence, return_tensors="pt")
15
16 with torch.no_grad():
17     outputs = model(
18         **inputs,
19         output_hidden_states=True,
20         output_attentions=True
21     )
22
23 print("input_ids shape:", inputs["input_ids"].shape)
24 print("last_hidden_state shape:", outputs.last_hidden_state.shape)
25 print("number of hidden_states:", len(outputs.hidden_states))
26
27 if outputs.attentions is not None:
28     print("number of attention tensors:", len(outputs.attentions))
29     print("attention layer 0 shape:", outputs.attentions[0].shape)
30 else:
31     print("No attention maps returned by this runtime/configuration.")
```

7.2 Visualización de una matriz de atención

```
1 if outputs.attentions is not None:
2     tokens = tokenizer.convert_ids_to_tokens(inputs["input_ids"][0])
3     layer = 0
4     head = 0
5     attention_matrix = outputs.attentions[layer][0, head].detach().cpu().numpy()
6
7     plt.figure(figsize=(7, 6))
8     plt.imshow(attention_matrix, aspect="auto")
9     plt.colorbar(label="attention weight")
10    plt.xticks(range(len(tokens)), tokens, rotation=90)
11    plt.yticks(range(len(tokens)), tokens)
12    plt.title(f"Attention - layer {layer}, head {head}")
```

```
13 plt.tight_layout()
14 plt.show()
```

7.3 Masked language modeling con top-k

```
1 mlm_model = EsmForMaskedLM.from_pretrained(model_name)
2 mlm_model.eval()
3
4 # En algunas versiones el tokenizer maneja el token de mascara internamente.
5 mask = tokenizer.mask_token
6 masked_sequence = "MKTAYIAKQRQISFVK".replace("Y", mask, 1)
7
8 masked_inputs = tokenizer(masked_sequence, return_tensors="pt")
9 mask_index = (masked_inputs["input_ids"] == tokenizer.mask_token_id).nonzero(as_tuple=True)
10 [1]
11
12 with torch.no_grad():
13     mlm_outputs = mlm_model(**masked_inputs)
14
15 logits = mlm_outputs.logits[0, mask_index, :]
16 topk = torch.topk(logits, k=5, dim=-1)
17
18 for score, token_id in zip(topk.values[0], topk.indices[0]):
19     token = tokenizer.convert_ids_to_tokens([token_id.item()])[0]
20     print(token, float(score))
```

8 Entregables

8.1 Entregable 1: notebook reproducible

Debe estar en un repositorio o archivo comprimido. Debe ejecutarse en orden. No se aceptan celdas rotas ni resultados sin explicación.

8.2 Entregable 2: informe técnico

Extensión sugerida: 12 a 20 páginas sin contar anexos. Debe contener:

1. portada con integrantes y enlace al repositorio;
2. resumen ejecutivo de máximo 300 palabras;
3. marco teórico;
4. explicación de ESM-2;
5. descripción de datos/secuencias;
6. metodología computacional;
7. inspección del código fuente;
8. resultados y visualizaciones;
9. discusión de usos reales;
10. limitaciones;

11. conclusiones;
12. referencias.

8.3 Entregable 3: defensa oral individual

La defensa durará entre 8 y 12 minutos por grupo, más preguntas. Cada estudiante debe responder individualmente al menos una pregunta técnica.

9 Rúbrica de evaluación

Criterio	Puntos	Evidencia esperada
Marco teórico sobre transformers, tipos de arquitectura y ESM-2.	15	Texto claro, con citas y relación explícita con el modelo usado.
Ejecución reproducible de ESM-2 preentrenado.	10	Notebook ejecutable, entorno declarado y salidas verificables.
Inspección de código fuente.	15	Identificación de embeddings, atención, capas, heads, MLM head y salida.
Experimentos con secuencia original, mutada, alterada y enmascarada.	15	Comparaciones, tablas, métricas y explicación de diferencias.
Visualización e interpretación de embeddings, hidden states y atenciones.	20	Matrices de atención o justificación técnica si no se retornan; gráficas interpretadas.
Discusión de usos reales y limitaciones.	10	Aplicaciones realistas y límites científicos claros.
Informe técnico y calidad comunicativa.	5	Redacción, estructura, figuras, referencias y reproducibilidad.
Defensa oral individual.	10	Capacidad de explicar sin leer y responder preguntas técnicas.

10 Criterios de no aprobación

Un trabajo puede reprobar aunque tenga buena presentación si incurre en alguno de estos casos:

- Solo usa una API o una celda de inferencia sin explicar arquitectura ni salidas internas.
- No evidencia ejecución real de ESM-2.
- No identifica componentes en código fuente.
- No diferencia encoder-only, decoder-only y encoder-decoder.

- Confunde embeddings iniciales con hidden states contextualizados.
- Afirma conclusiones biomédicas no justificadas.
- Presenta texto generado sin comprensión en defensa individual.

11 Preguntas de defensa

El profesor puede usar estas preguntas:

1. ¿Por qué ESM-2 es encoder-only y no GPT?
2. ¿Qué es un token en ESM-2?
3. ¿Qué diferencia hay entre `input_ids`, `embeddings` y `last_hidden_state`?
4. ¿Qué forma tiene una matriz de atención y qué representa cada eje?
5. ¿Qué son Query, Key y Value?
6. ¿Por qué ESM-2 puede mirar ambos lados de una posición enmascarada?
7. ¿Por qué no se puede interpretar automáticamente una cabeza de atención como prueba causal biológica?
8. ¿Cómo se obtiene un embedding global de proteína?
9. ¿Qué cambio observaron entre la secuencia original y la mutada?
10. ¿Qué uso real de ESM-2 les parece más viable y por qué?

12 Cronograma sugerido

Semana	Actividad	Peso
Semana 1	Lectura de marco teórico, instalación, carga del modelo y primera inferencia.	10 %
Semana 2	Extracción de tokens, embeddings, hidden states, atenciones y lectura de código fuente.	25 %
Semana 3	Experimentos con secuencia original, mutada, alterada y enmascarada.	30 %
Semana 4	Informe técnico, discusión de usos reales, limitaciones y defensa oral.	35 %

13 Anexo A: plantilla de estructura del informe

1. Portada.
2. Resumen ejecutivo.
3. Introducción.
4. Marco teórico.
5. ¿Qué es ESM-2?
6. Metodología computacional.
7. Inspección del código fuente.
8. Resultados.
9. Discusión.

10. Usos reales de ESM-2.
11. Limitaciones.
12. Conclusiones.
13. Referencias.
14. Anexos de código y salidas.

14 Anexo B: secuencias de ejemplo

Estas secuencias son didácticas y pueden reemplazarse por secuencias reales suministradas o aprobadas por el profesor.

```
1 Original:
2 MKTAYIAKQRQISFVKSHFSRQDILD
3
4 Mutada, cambio Y -> F:
5 MKTAFIAKQRQISFVKSHFSRQDILD
6
7 Alterada por inversion parcial:
8 DLIDQRSFHSSKVFSIQRQKAIYATKM
```

15 Anexo C: lectura mínima recomendada

- Vaswani et al. (2017): arquitectura transformer y mecanismo de atención.
- Devlin et al. (2018): entrenamiento encoder-only bidireccional con masked language modeling.
- Radford et al. (2018): transformer decoder para generación autoregresiva.
- Lin et al. (2023): modelos de lenguaje de proteínas y predicción estructural.
- Documentación de Hugging Face sobre ESM y salidas de modelos (Hugging Face, 2026a, 2026b).

Referencias

- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2018). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *arXiv preprint arXiv:1810.04805*. <https://arxiv.org/abs/1810.04805>
- Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., & Houlsby, N. (2020). An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. *arXiv preprint arXiv:2010.11929*. <https://arxiv.org/abs/2010.11929>
- Hugging Face. (2026a). ESM model documentation [Consultado el 11 de mayo de 2026]. https://huggingface.co/docs/transformers/en/model_doc/esm
- Hugging Face. (2026b). Transformers model outputs: hidden states, attentions and logits [Consultado el 11 de mayo de 2026]. https://huggingface.co/docs/transformers/en/main_classes/output
- Hugging Face and Meta AI. (2026). facebook/esm2_t6_8M_UR50D model card [Consultado el 11 de mayo de 2026]. https://huggingface.co/facebook/esm2_t6_8M_UR50D
- Lin, Z., Akin, H., Rao, R., Hie, B., Zhu, Z., Lu, W., dos Santos Costa, A., Fazel-Zarandi, M., Sercu, T., Candido, S., & Rives, A. (2023). Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science*, 379(6637), 1123-1130. <https://doi.org/10.1126/science.ade2574>
- Meta Fundamental AI Research Protein Team. (2026). Evolutionary Scale Modeling: pretrained language models for proteins [Consultado el 11 de mayo de 2026]. <https://github.com/facebookresearch/esm>
- Radford, A., Narasimhan, K., Salimans, T., & Sutskever, I. (2018). Improving Language Understanding by Generative Pre-Training. https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention Is All You Need. *Advances in Neural Information Processing Systems*. <https://arxiv.org/abs/1706.03762>